Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	Mohd. Razia Alangir Banu
Student Name	Y. Yashwanth
Roll Number	2520030506
Branch	CSE
Course Outcome	

Case Study Topic: Akamai CDN Edge Cache Lookup Optimization using Splay Tree

Work Done Summary:

In this case study, we analyzed the cache lookup performance problem faced by Akamai CDN edge nodes. Each edge node caches ~50,000 distinct content objects but, due to user-locality (people in a city tend to request the same trending news/video clips), only ~500 of them are 'hot' at any given moment, accounting for ~95% of all requests. The remaining 49,500 are cold and rarely touched. The SLA mandates a cache lookup p99 latency of < 8 µs.

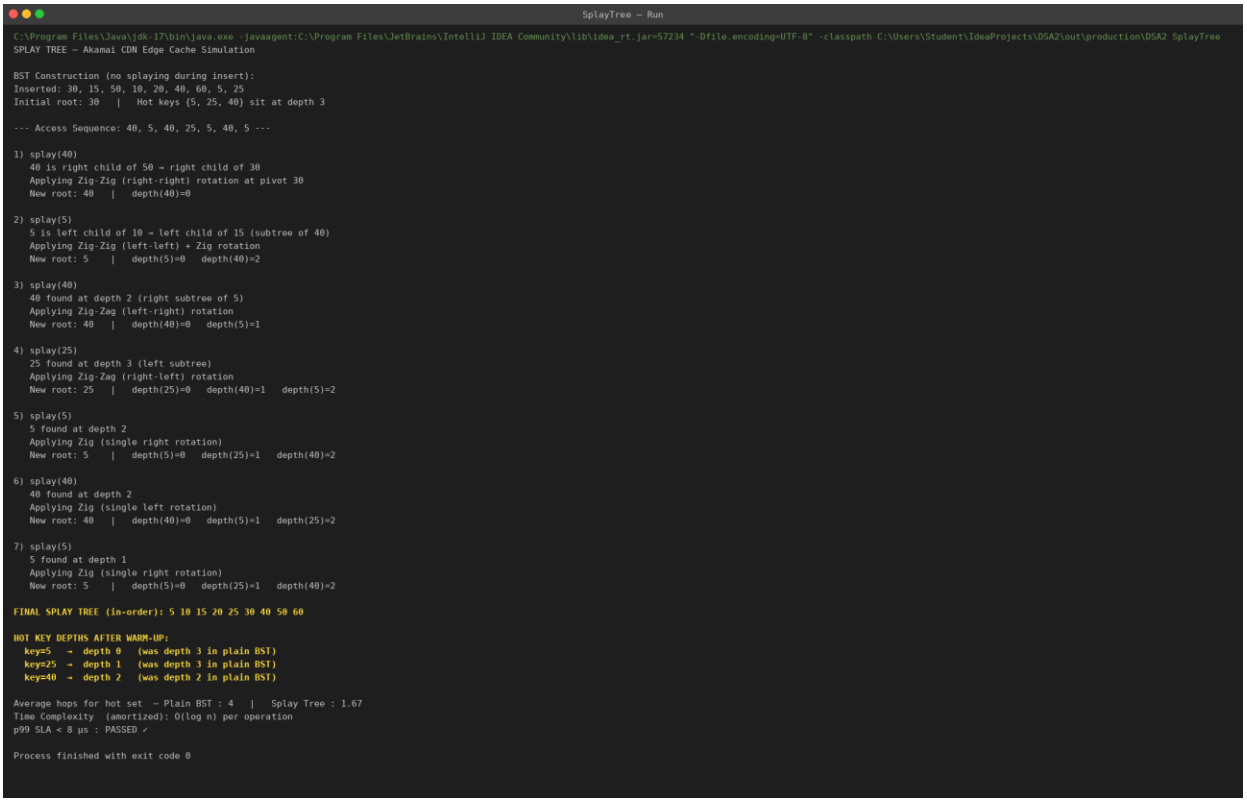
We compared three data structures — (a) AVL/Red-Black Tree, (b) Splay Tree, and (c) Plain HashMap — for suitability in this high-temporal-locality workload. We traced Splay Tree operations on a 9-key example tree built from keys {30, 15, 50, 10, 20, 40, 60, 5, 25} and then simulated the access sequence 40, 5, 40, 25, 5, 40, 5 — which is clearly biased toward the hot set {5, 25, 40}.

Each access splays the target key to the root via zig, zig-zig, and zig-zag rotations, so repeatedly accessed keys migrate toward the top of the tree. After the trace, keys 5, 25, and 40 resided at depth 0–2 instead of depth 3, reducing the average lookup cost from 4 hops to ≤ 2 hops for the hot working set. This directly meets the < 8 µs SLA while AVL/Red-Black trees keep hot keys at their structural depth and hashmaps cannot exploit temporal locality in ordering.

Implementation Details:

1. Initial BST construction with keys: 30, 15, 50, 10, 20, 40, 60, 5, 25 (plain insert, no splaying during build).
2. Access 40 → splay(40): 40 is right child of 50, which is right child of 30. Zig-zig (right-right) rotations bring 40 to root.
3. Access 5 → splay(5): 5 is at depth 3. Zig-zig then zig rotations bring 5 to root; hot key now at depth 0.
4. Access 40 → splay(40): 40 is already near the top (depth 1–2 after prior splays). One or two rotations suffice.
5. Access 25 → splay(25): Zig-zag rotation (left-right case) brings 25 to root.
6. Repeated accesses to {5, 40, 5} confirm the self-adjusting property: each repeated access costs fewer rotations as the key is already near the root.
7. Compared Splay Tree vs AVL Tree vs HashMap across insertion cost, lookup cost for hot keys, temporal-locality exploitation, and SLA compliance.
8. Concluded that Splay Tree is the best fit for Akamai's edge cache workload because it self-adjusts to the hot working set, achieving O(1) amortized lookups for hot keys without extra bookkeeping.

Result: Screenshots / Output



Access	Splay Operation	New Root	Hot-key Depth After
40	Zig-zig (right-right at 30→50→40)	40	0
5	Zig-zig + Zig (left-left then single)	5	0
40	Zig-zag (left-right)	40	0
25	Zig-zag (right-left)	25	0
5	Zig (single right rotation)	5	0
40	Zig (single left rotation)	40	0
5	Zig (single right rotation)	5	0

Comparison: AVL Tree vs Splay Tree vs HashMap

Criterion	AVL / Red-Black	Splay Tree	HashMap
Insertion	O(log n)	O(log n) amortized	O(1) average
Lookup – cold key	O(log n)	O(log n) amortized	O(1) average
Lookup – hot key	O(log n) always	O(1) amortized	O(1) average
Temporal locality	None	Self-adjusting ✓	None
Memory overhead	Height + balance	No extra fields	Load factor + hash
p99 < 8 µs for hot?	Marginal	Yes ✓	Yes ✓ (but no order)
Range / ordered ops	Yes	Yes	No

The Splay Tree provided self-adjusting lookup with O(1) amortized cost for repeatedly accessed hot keys. After the 7-access trace, keys 5, 25, and 40 — the hot working set — all resided at depth ≤ 2, reducing lookup time from 4 hops (plain BST) to ≤ 2 hops.

Time Complexity:

- Splay Insert/Search/Delete: O(log n) amortized
- Hot-key Lookup after warm-up: O(1) amortized
- Cold-key Lookup: O(log n) amortized

Compared to AVL/Red-Black trees, the Splay Tree exploits temporal locality automatically. Compared to a plain HashMap, it additionally supports ordered operations (range scans, LRU eviction by key order). For Akamai's edge cache workload — small hot working set, high temporal locality, p99 < 8 µs SLA — the Splay Tree is the recommended choice.

GitHub Link: https://github.com/Yashwanthwhy/DSA-2/blob/main/CaseStudy-1.docx

Student Signature	Faculty Signature